

NANYANG TECHNOLOGICAL UNIVERSITY

Example Class

Prof Douglas Maskell

asdouglas@ntu.edu.sg

N4-02c-98

School of Computer Engineering

NTU

Course Site: https://ntulearn.ntu.edu.sg/

Schedule 2023

Week	Topic	Tutorial	Lecture 1 (Monday 8:30)	Lecture 2 (Wednesday 8:30)
1	Mod 1 (Intro, Review Verilog, Design Flow, IP)	No Tut	Introduction	Not used
2	Mod 2 (Verification & Verilog Testbenches)	Tut 1	Not Used	Not used
3	Mod 3 (FPGA Architecture and Synthesis)	Tut 2	Not used	Not Used
4	Mod 4 (Arithmetic on FPGAs)	Tut 3	Not used	**EC1** (Mod 1 & 2)
5	Mod 5 (Timing and Pipelining)	Tut 4	Not Used	**EC2** (Mod 3 & 4)
6	Mod 5 (Timing and Pipelining) continued	Tut 5	Not used	Not used
7	Mod 7 (**eLearning** Buses & interfacing)	No Tut	Not used	**QUIZ**
Recess				
8	Mod 8 (uP Case Study)	No Tut	Not Used	**EC3** (Mod 5 & 7)
9	Mod 8 (uP Case Study)	Tut 7	Not used	Not used
10	Mod 9 (Asynchronous Design)	Tut 8	Not used	**EC4** (Mod 8)
11	Mod 9 (Asynchronous Design)	Tut 9	Not Used	Not used
12	--	Tut 10	Not used	**EC5** (Mod 9)
13	--	Not used	Not used	Not used

The yellow shaded regions are for Example Classes (which are in addition to the LAMS-based lectures).
The green shaded region is for a class quiz (details will be provided later).
There is no Tutorial 6 as this has been integrated into the eLearning module.
The current Lab schedule can be accessed from: https://freeaccess.scse.ntu.edu.sg/scselabs

This may change slightly throughout the semester
Individual Tutors are likely to change the tutorial timing.

Assessment

- The course assessment is composed of two parts:
  - The Final Exam (60%), and
  - Coursework (40%)
    - Of the 40 marks allocated to Coursework
      - 24 marks will be for the quiz (held in week 7)
        - » Past quizzes and solutions are on NTUlearn (in “Content” folder)
      - 16 marks will be for your performance in the labs
        - » Experiential learning
        - » Each lab (Lab 2-Lab5) is allocated 4 marks
        - » The lab must be completed in a single lab session
          - Make-up classes are only allowed for valid reasons (such as illness with an MC). Absence due to other activities will not be considered.
          - You may want to use make-up sessions BEFORE the lab so that you can easily complete the lab and the assessment in the time provided.
        - » Lab 1 is not assessable

Examination

- The Final Exam represents 60% of the total assessment
  - The exam consists of two parts
    - Multiple choice section (~50-60%)
    - Two/three compulsory questions
  - Check the April/May 2019 CE2003 exam (or later) for the format.
    - The format earlier than 2019 is different (no MCQ), but the questions are still relevant.
    - In 2020 there was no exam. Instead, 100% coursework due to Covid SMMs.



NANYANG TECHNOLOGICAL UNIVERSITY

Example Class

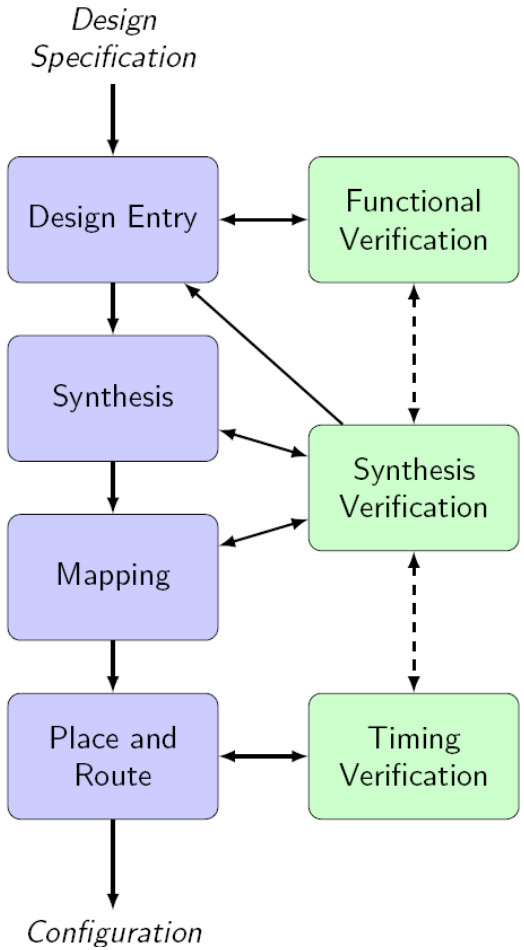
Module 1-2

Summary

- Module 1: Review
  - Design flow; IP, Verilog recap
- Module 2: Verification and Testing
  - Verilog testbenches

Design Flow

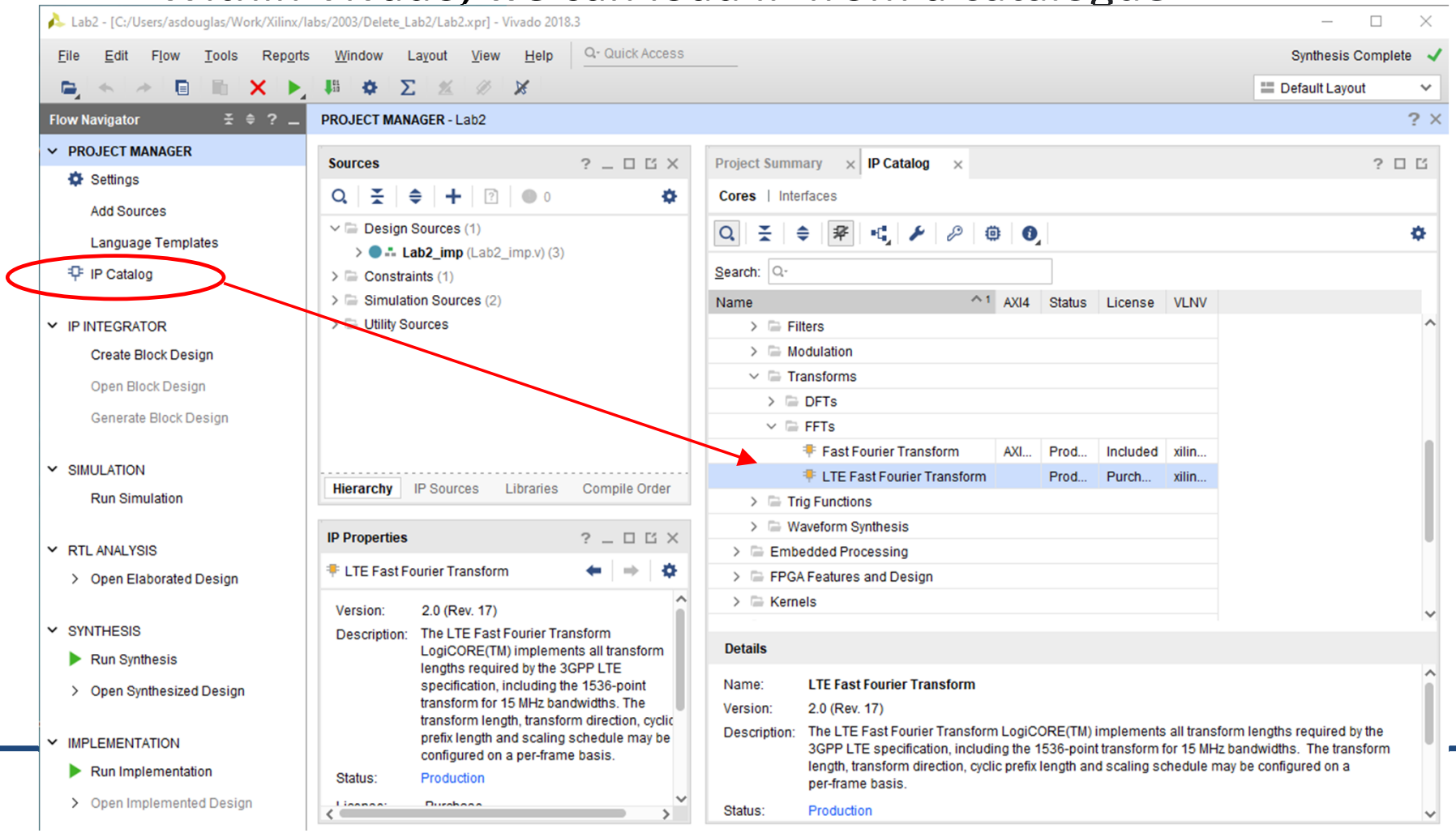
- We introduced digital design and the design flow



But, more on that in the next module

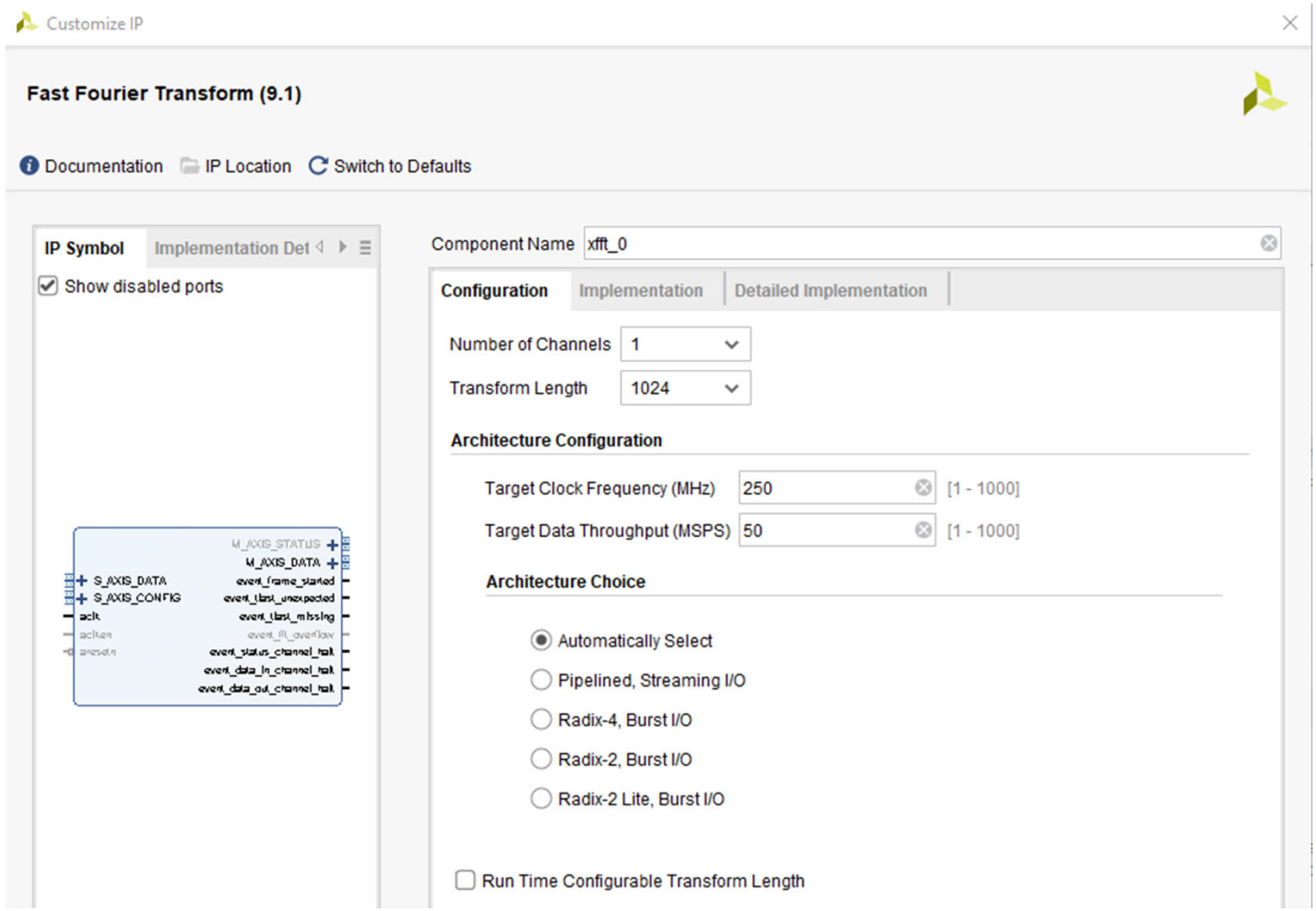
Intellectual Property

Within Vivado, we can load IP from a catalogue



Intellectual Property

Upon selecting an IP block, you customise it to your needs:



Verilog Review

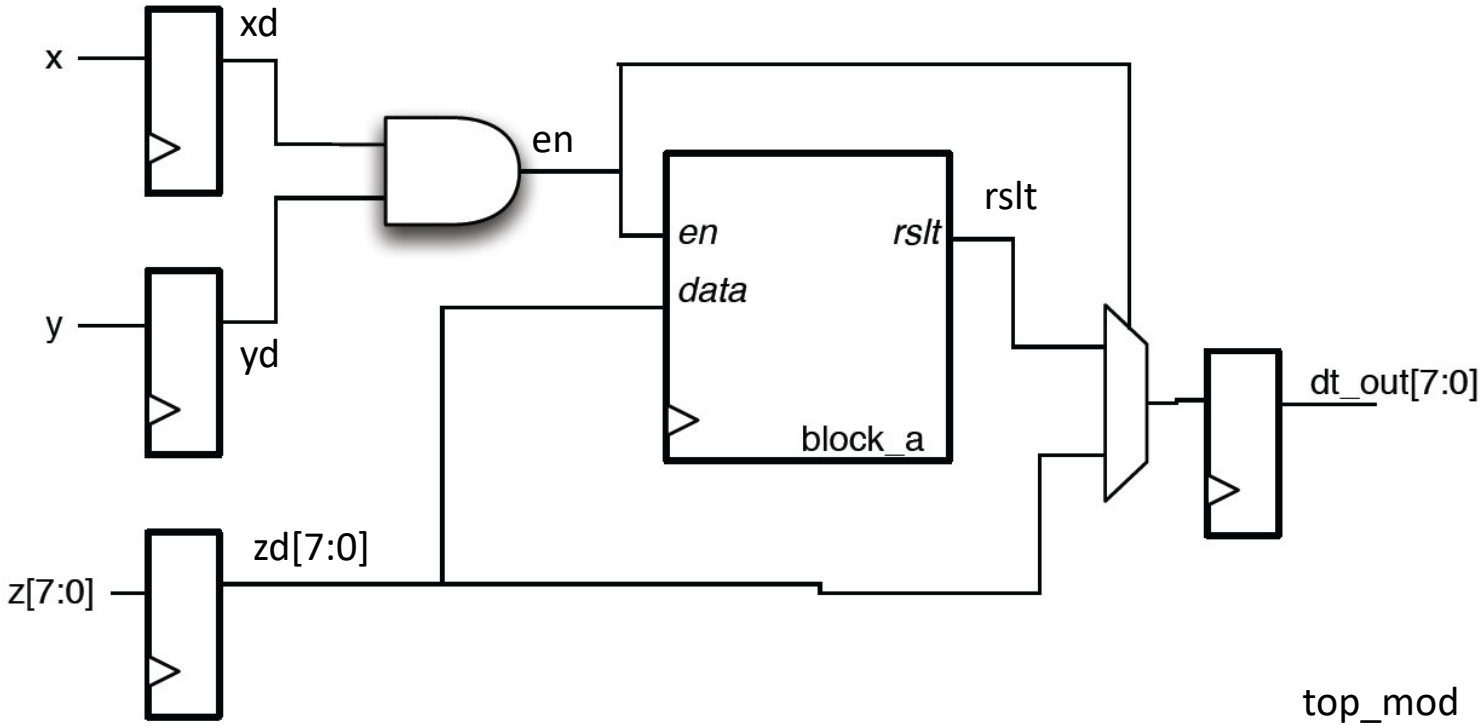
- We basically went through synthesisable Verilog (a subset of Verilog).
  - Modules and module instantiation.
    - The need to declare reg and multi-bit wires
  - Continuous assignment (**assign** x=y;)
  - Combinational always blocks (**always** @ *)
    - Use blocking assignments (=)
    - Ensure **all** paths assign to **all** outputs. A default assignment at the top helps
    - Never use a continuous **assign** inside an always block
  - Synchronous always blocks (**always** @ (posedge clk))
    - Use non-blocking assignments (<=). The "<=" will generate a register.
    - Should include a rst
  - **Do not** assign to the same signals from multiple blocks or assign statements

Remember, when you make an assignment from within an **always** block, the signal must be declared as **reg**.

Verilog Recap: Homework

Write the verilog code for the following circuit:

- Note: I am assuming that rslt is 8-bit and am using the signals as named.

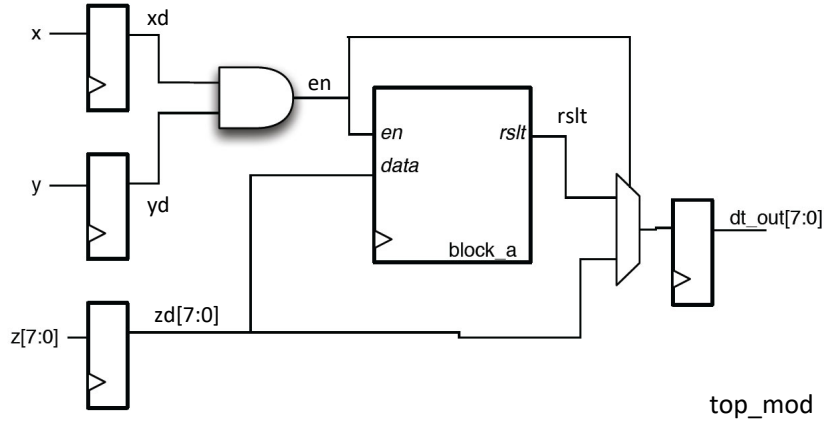


```
module top_mod (input x, y, clk, rst, input [7:0] z, output reg [7:0] dt_out);
    reg xd, yd;
    reg [7:0] zd;
    wire [7:0] rslt;
    block_a u1 (.en(en), .data (zd), .clk(clk), .rst(rst), .rslt(rslt));
    assign en = xd & yd;
endmodule
```

Declare the internal signals. Signals xd, yd and zd must be declared as **reg** as they will be assigned to from within an **always** block. The signal rslt is from a module instantiation, so it is a **wire**.

Instantiale block_a (note the clk and rst) and use an **assign** to generate en.

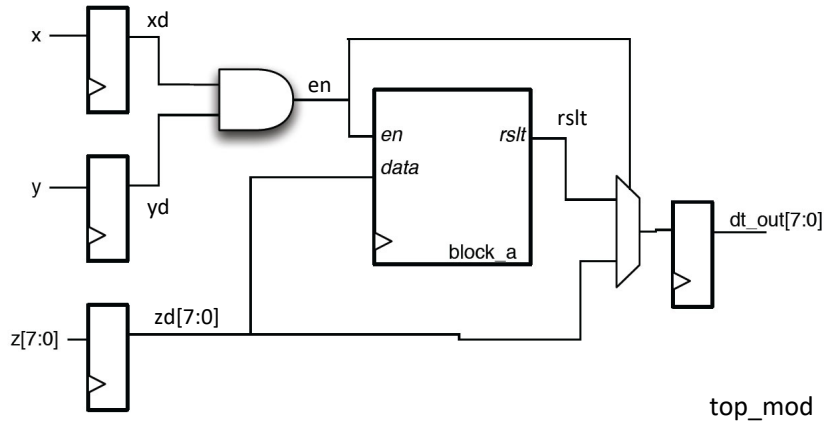
Note that I could have eliminated the **assign** statement and used:
block_a u1 (.en(xd & yd), .data (zd), .clk(clk), .rst(rst), .rslt(rslt));



```
module top_mod (input x, y, clk, rst, input [7:0] z, output reg [7:0] dt_out);
    reg xd, yd;
    reg [7:0] zd;
    wire [7:0] rslt;
    block_a u1 (.en(en), .data (zd), .clk(clk), .rst(rst), .rslt(rslt));
    assign en = xd & yd;
    always @ (posedge clk) begin
        if (rst) begin xd <= 0; yd <= 0; zd <= 0; dt_out <= 0; end
        else begin
            xd <= x;
            yd <= y;
            zd <= z;
            if (en) dt_out <= rslt;
            else dt_out <= zd;
        end
    end
endmodule
```

Lastly, use a non-blocking assignment for dt_out. We could have also used a conditional assignment, for just the mux, but outside of the always block.

Use non-blocking assignments for xd, yd and zd, so that registers are generated.





NANYANG TECHNOLOGICAL UNIVERSITY

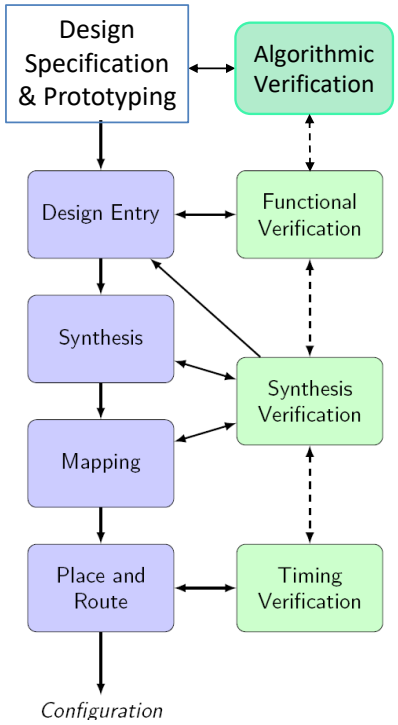
Example Class

Module 2

Verification and Testing

Introduction to Verification

Verification of a system can be considered at a number of levels:



- **Algorithmic verification:** at the highest level, we want to know that the algorithm we have selected is suitable for the desired application, even before we think about implementation
- **Functional verification:** once we have designed the hardware, we want to ensure that the architecture correctly implements the algorithm
- **Synthesis verification:** we need to ensure the architecture we have designed is fully synthesisable and implementable on the desired target platform
- **Timing verification:** once the circuit is synthesised, placed & routed, does it meet our timing constraints?

Verilog Test Benches

- A self-contained module, typically with no inputs or outputs
- Instantiates the ***unit under test*** – the design we want to verify
- Contains a number of blocks that interact with the *unit under test*
  - Data and control signal generators; Clock generators; Data and status signal monitors:
- Introduced the **initial** block: Like an always block BUT only runs once.
  - Non-synthesisable.
  - Can be more flexible than with synthesisable Verilog. Can assign to same signal from multiple blocks
- Introduced the concept of time steps. #10 means wait for 10-time steps.
  - Non-synthesisable
  - timescale directive (**‘timescale** 1 ns / 100 ps)
- Looked at various system calls for inputting and outputting simulation data
  - $display(); $write(); Various file calls: $fopen(), $fscanf(); $fwrite(); $fclose(), etc.
  - Saw that these were very similar to the C-language equivalent, including formatting.

Verilog Test Benches

- Saw that simulation signals could actually have four possible values
  - Covered unknown (X) and high impedance (Z); === and !== operators; **casex** and **casez** statements.

Case Equality operands are compared bit by bit, with '0' padding if they are not the same length

Values with x and z must match for a True result.

```
4'bx001 ===  4'bx001      = 1
4'bx0x1 ===  4'bx001      = 0
4'bz0x1 ===  4'bz0x1      = 1
4'bz0x1 ===  4'bz001      = 0
4'bx0x1 !==  4'bx001      = 1
4'bz0x1 !==  4'bz001      = 1
4'b101x === 4'b101x       = 1
4'b101x !==  4'b101x      = 0
```

Result is x for **Equality, Logical and Bitwise**, with x or z.

```
4'b1100 == 4'b1100        = 1
4'b1100 != 4'b1100        = 0
4'b1100 == 4'b1z10        = 0    // False due of bit 2.
4'b1100 != 4'b100x        = 1    // True due to bit 3.
```

Equality Operators (continued)

```
4'b101x == 4'b101x   = x
4'b101x != 4'b101x   = x
```

Logical Operators

```
1'b1 && 1'b1 = 1;   1'b1 && 1'b0  = 0;    1'b1 && 1'bx = x;
1'b1 || 1'b0  = 1;   1'b0 || 1'b0   = 0;    1'b0 || 1'bx   = x;
```

Bitwise operators

```
~4'b0001 = 1110;   ~4'bx001 = x110;    ~4'bz001 = x110;
4'b0001 &  4'b1001   = 0001
4'b1001 &  4'bx001   = x001
4'b1001 &  4'bz001   = x001
8'b1010xzxz & 8'b10010011 = 8'b100000xx;  bits 3,4 are '0' as 0&x = 0
8'b1010xzxz | 8'b10010011  = 8'b1011xx11;  bits 1,2 are '1' as 1|x = 1
8'b1010xzxz ^ 8'b10010011  = 8'b0011xxxx;
8'b1010xzxz ~^ 8'b10010011 = 8'b1100xxxx;
~ 8'b1010xzxz = 8'b0101xxxx;
```

Verilog Test Benches

- Saw that simulation signals could actually have four possible values
  - Covered unknown (X) and high impedance (Z); === and !== operators; **casex** and **casez** statements.
- Saw how to create clock signals and pulses (one off or repeating)
- Introduced functions and tasks.
  - These can be synthesisable

- Discussed testing strategies (against a software implementation, using a software inverse, etc)
- Saw that comprehensive testing usually takes a lot more time and effort than designing the circuit.

Solns to previous Exams (CE2003)

Q1 S2 16/17

1. A 4-bit Verilog up/down counter (*updn_cnt*) increments by three if the up/down control signal, ***dir***, is zero and decrements by two if ***dir*** is one.

   (a) Write the Verilog code to implement the *updn_cnt* module. The counter should always start counting from zero and the 4-bit counter signal, ***cout***, is an output from the module.

   (b) The ***dir*** signal is one when the counter first counts (that is, at the first actual count cycle) and then subsequently toggles just before count cycle 2, 4, 5, and 7. The simulation finishes at the end of count cycle 9.

   Write the Verilog code to implement a testbench (called *updn_cnt_tb*) to simulate the counter module. Your code should declare the testbench internal signals, instantiate the module under test, and include any necessary stimuli. Remember to add the **$finish** statement. You should assume that a count cycle is 10 timesteps.

   (c) Draw a timing diagram showing all signals in the module port list. You may ignore any propagation delay.

Q1 S2 16/17 (cont)

Q1(a) 4-bit counter (*updn_cnt*) increments by 3 if ***dir*** = 0 and decrements by 2 if ***dir*** = 1. The counter should always start counting from zero and the 4-bit counter signal, ***cout***, is an output from the module.

```
module updn_cnt (input clk, rst, dir, output reg [3:0] cout);
    always @ (posedge clk)
    begin
        if (rst) cout <=0;
        else if (dir) cout <= cout-2;
            else cout <= cout+3;
    end
endmodule
```

Q1 S2 16/17 (cont)

Q1(b) The ***dir*** signal is 1 when the counter first counts and toggles just before count cycle 2, 4, 5, and 7. The simulation finishes at the end of count cycle 9. Write the code to implement a testbench (called *updn_cnt_tb*) to simulate the counter module.

```
module updn_cnt_tb();
    // Instantiate the module I/O
    reg clk, rst, dir;
    wire [3:0] cout;
    // Instantiate the Unit Under Test (UUT)
    updn_cnt u1 (.clk(clk), .rst(rst), .dir(dir), .cout(cout));
    initial
    begin
        // Initialize Inputs
        clk = 0; rst = 1; dir = 1;
        // Add stimulus here
        #10 rst = 0;
        #10 dir = 0;
        #20 dir = 1;
        #10 dir = 0;
        #20 dir = 1;
        #20 $finish;
    end
    always #5 clk = !clk;
endmodule
```

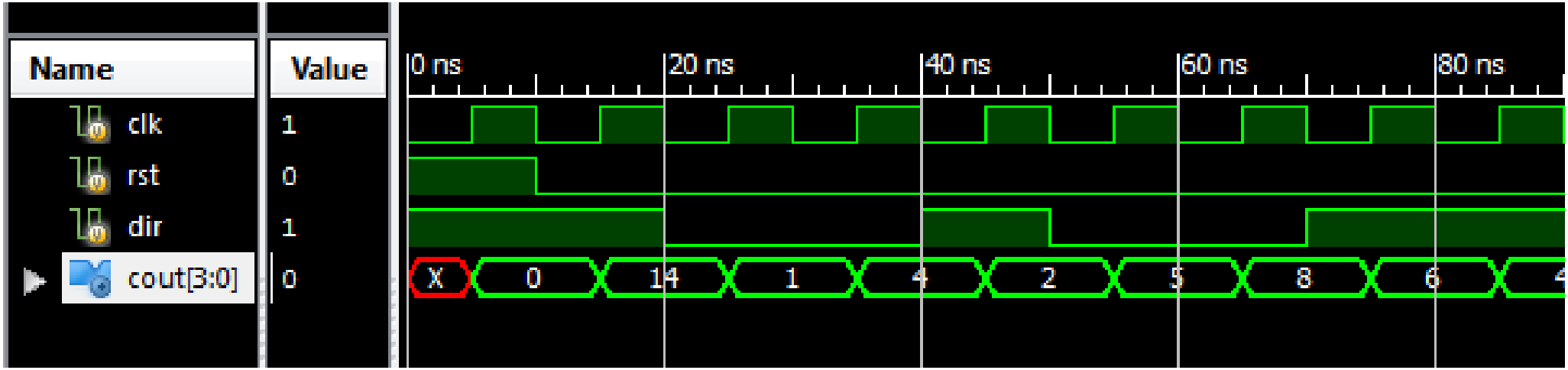
```
module updn_cnt (input clk, rst, dir, output reg [3:0] cout);
```

1st rising edge @ 5ns

Q1 S2 16/17 (cont)

Q1(c) Draw a timing diagram showing all signals in the module port list. You may ignore any propagation delay.

Remember: At t=0 clk = 0; rst = dir = 1; At t=10 rst = 0; At t=20 dir = 0; At t=40 dir = 1; At t=50 dir = 0; At t=70 dir = 1; and at t=90 we finish.



Solns to previous Exams

Q1 S1 17/18

1. (b) Write the Verilog code for a module that implements a 4-bit synchronous counter with output *count*. When the input *short* is asserted the counter's maximum value is limited to 4. If *short* is asserted when the counter's value is higher, the next value should be zero.

(c) Write a testbench for the module in Q1(b). *Reset* is high for 10ns and goes low on the negative clock edge. *Short* is initially low and goes high 60 ns after *reset* goes low. The simulation ends after 145ns. Using a 10 ns clock, sketch the input signals and the expected output.

Solns to previous Exams

Q1 S1 17/18

1. (b) Write the code that implements a 4-bit synchronous counter with output *count*. When the input *short* is asserted the counter's maximum value is limited to 4. If *short* is asserted when the counter's value is higher, the next value should be zero.

So we want to count: Short=0 {0, 1, 2, …, E, F, 0, 1…}. If short=1 {8, 0, 1, 2, 3, 4, 0, 1, 2, 3, 4, 0, …}

```
module cnt_4 (input clk, rst, short, output reg [3:0] count);
        always @ (posedge clk)
        begin
            if (rst) count <= 0;
            else if (short && (count >= 4) count <= 0;
                    else count <= count+1;
        end
endmodule
```

Solns to previous Exams

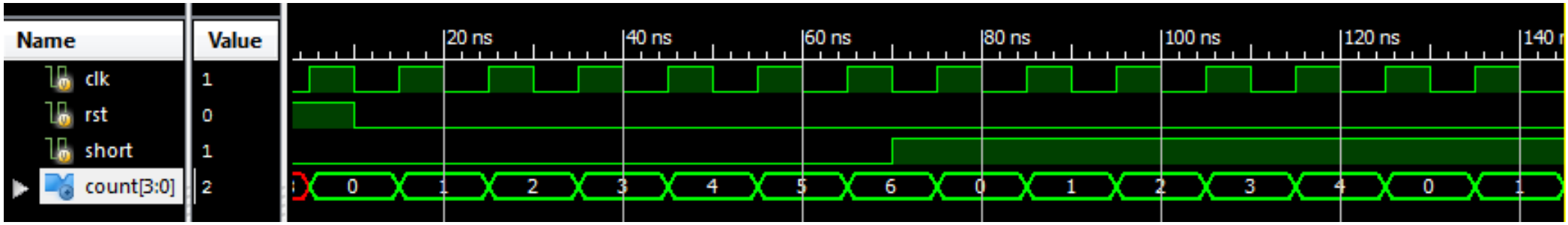
Q1 S1 17/18 (c)Write a testbench for Q1(b). Reset is high for 10ns and goes low on the negative clock edge. Short is initially low and goes high 60 ns after reset goes low. The simulation ends after 145ns. Using a 10 ns clock. Sketch the I/O signals.

```
module cnt_4_tb();
    reg clk, rst, short;
    wire [3:0] count;
    cnt_4 uut (.clk(clk), .rst(rst), .short(short), .count(count));
    initial begin
        clk = 0; rst = 1; short = 0;
        #10 rst = 0;
        #60 short = 1;
        #75 $finish;
    end
    always #5 clk = !clk;
endmodule
```

Solns to previous Exams

Q1 S1 17/18 (c) cont

```
initial begin
    clk = 0; rst = 1; short = 0;
    #10 rst = 0;
    #60 short = 1;
    #75 $finish;
end
always #5 clk = !clk;
```



Soln to previous Exams

Q1 S2 17/18 (a) Digital designers describe circuits using hardware description languages (HDLs), like Verilog, or other tools that translate higher level descriptions into HDL. This is usually referred to as the *Design Entry* phase. The code is then processed through a sequence of steps. Write down the three steps in the FPGA design flow in order, and then briefly describe what happens in each step.

(9 marks)

- Synthesis: Verilog HDL is converted into a netlist of primitives. This is iterative with verification/simulation.
  - Mapping: Netlists are mapped to the target technology's primitives. This step is usually included with synthesis in modern EDA tools.
- Placement: The primitives are placed to locations on the target device.
- Routing: The primitives (placed in the previous step) are connected as per the netlist. The router performs this as an iterative process trying to improve the routing results.
  - After place and route, timing verification/simulation is carried out.

End of EC1